// while you find a seat

Get the code

Scan now. Follow along later.



github.com/danvega/spring-cleaning

Spring Cleaning starts soon

- ~~spring-retry~~
- ~~spring-cloud-starter-openfeign~~
- ~~VersionFilter.java~~

Spring Cleaning

What Boot 4 lets you delete.

Dan Vega · J-Fall 2026

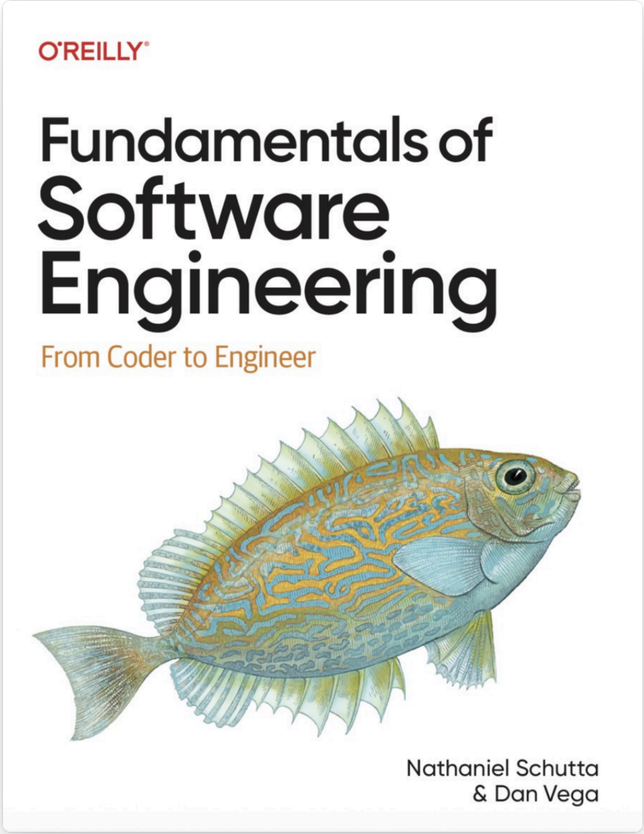
$ whoami

www.danvega.dev

Dan Vega

Spring Developer Advocate, Broadcom

+ Java Champion
+ 25+ years building software
+ Co-wrote this book with Nate Schutta
+ Co-host Spring Office Hours
+ Cleveland, OH. Husband and father.



// the elephant in the room

AI vulnerabilities

Let's talk about it first.

security reports to the Spring team, per month

6.5

The old normal.

Spring blog, June 1, 2026

security reports to the Spring team

6.5 a month, the old normal

482

In April 2026 alone.

Spring blog, June 1, 2026

// not just spring

It's hitting all of open source

Jan 2026	**curl ends its bug bounty to stop a flood of AI reports.** The Register, Jan 21, 2026
May 2026	**Linus Torvalds: the kernel security list is "almost entirely unmanageable."** Infosecurity Magazine, Sept 2, 2026
Oct 2026	**Google pauses its open source bug bounty. Most new reports aren't valid.** Help Net Security, Oct 5, 2026

// the real story

AI made finding vulnerabilities cheap.



It is strongly recommended to upgrade as soon as possible to the latest patches.

The Spring team, June 2026

You can patch faster.
Or you can have less to patch.

// when you can't delete it yet

TrueSource™
by Broadcom

Trusted Artifacts

SLSA Level 3 verified libraries for Java, Python and NPM, plus Bitnami Secure Images.

Data Services

Validated distributions and enterprise support for PostgreSQL, RabbitMQ, MySQL and Valkey.

Spring Enterprise

Secure builds, 24x7 support and day-zero CVE-only patches for Spring and its whole dependency tree.

+ Application Advisor Applies those patches across your apps, or plans the move to Boot 4.

// where you stand

Free support, by version

- ~~3.5~~ ~~ended Jun 30, 2026~~
- 4.0 ends Dec 31, 2026
- 4.1 until Jul 31, 2027 latest
- 4.2 due in November

Your app: which one?

// why delete anything

+ Agents are great at adding code.

- Not so great at deleting code.

Don't know what's built in?
You can't prompt for it.
You can't review it.

// today

Agenda

01 Boot cleaned house first

02 What you can delete

03 Small things, quietly useful

04 The upgrade plan

Part 1

Boot cleaned house first

modular auto-config · jackson 3

// part 1

Modularization

One jar became many.

// spring-boot-autoconfigure.jar

The jar kept growing

2014

185 KB

Boot 1.0, the first release

→

2025

2 MB

Boot 3.5. One big monolith, in every app.

// spring-boot-autoconfigure.jar

Boot 4 split up its auto-configuration

Boot 3.5

156

auto-configurations, all in this one jar

→

Boot 4.1

12

left in that jar. The rest moved into about 100 focused modules.

```
// boot did its own spring cleaning
```

- **Boot 3 configured everything it could find.**

+ **Boot 4 configures what you ask for.**

// boot did its own spring cleaning

One jar became many

```
// your classpath
- spring-boot-autoconfigure     // 156 auto-configs, one jar
+ spring-boot-autoconfigure     // 12, just the core
+ spring-boot-webmvc
+ spring-boot-jackson
+ spring-boot-flyway
+ spring-boot-h2console
+ ...                           // about 100 modules
```

// modular auto-configuration

Name what you use

// pom.xml

```
- spring-boot-starter-web
+ spring-boot-starter-webmvc
- flyway-core                    <!-- just worked in Boot 3 -->
+ spring-boot-starter-flyway
+ spring-boot-h2console          <!-- just worked in Boot 3 -->
  h2                             <!-- the database, unchanged -->
```

// modular auto-configuration

It fails quietly

- ! Flyway: no error. Your migrations just don't run.
- ! H2 console: a 404
- ! @WebMvcTest needs spring-boot-starter-webmvc-test
- ! @MockBean is gone. Use @MockitoBean.

// escape hatch

spring-boot-starter-classic

Turns the old behavior back on. Get green,
then move to focused starters one at a time.

// part 1

Jackson 3

The JSON library under everything.

// jackson 3

What's new in the box

+ Immutable, builder-based configuration
+ Unchecked exceptions, better for lambdas and streams
+ Boot auto-configures a JsonMapper bean
+ ISO-8601 dates out of the box
+ New tools.jackson packages
+ Jackson 2 and 3 side by side

// jackson 3

Inject the auto-configured mapper

```
// DataLoader.java
@Component
class DataLoader implements CommandLineRunner {
  private final JsonMapper mapper;
  private final ResourceLoader loader;
  DataLoader(JsonMapper mapper, ResourceLoader loader) { ... }
  public void run(String... args) throws Exception {
    var json = loader.getResource(DONUTS_JSON).getInputStream();
    donuts = mapper.readValue(json, new TypeReference<>() {});
  }
}
```

// jackson 3

What bites

- ! The databind packages moved to tools.jackson
- ! Mappers are immutable. Configure them with a builder.
- ! Dates are ISO-8601 strings, even in mappers you build
- ! A null for a primitive field now fails

```
// need more time?
```

```
spring.jackson.use-jackson2-defaults: true
```

A bridge, not a home. It only covers Boot's own mapper.

LIVE

```
$ git checkout demo-01-start
```

It worked yesterday

Boot 3.5 to 4.1, one break at a time.

Part 2

What you can delete

whatever Boot 4 now does for you

// part 2

Spring Retry

Retries are built in now.



// delete: spring-retry

Spring Retry goes

-1 dependency

```
- @EnableRetry
- @Retryable(retryFor = ApiException.class,
-    maxAttempts = 4, backoff = @Backoff(
-       delay = 1000, multiplier = 2))
+ @EnableResilientMethods
+ @Retryable(maxRetries = 3,
+    includes = ApiException.class,
+    delay = 1000, multiplier = 2)
```

// delete: spring-retry

Spring Retry goes

-1 dependency

```
+ @EnableResilientMethods
+ @Retryable(maxRetries = 3,
+    includes = ApiException.class,
+    delay = 1000, multiplier = 2)
```

```
// watch the count
```

```
- maxAttempts = 4
+ maxRetries  = 3
```

Spring Retry counts the first call.
Framework 7 counts only the retries.

// framework 7

Retry a flaky call with one annotation

```
// SpeakerService.java
@Service   // plus @EnableResilientMethods on the app
class SpeakerService {
  @Retryable(
      includes = HttpServerErrorException.class,  // only retry a 5xx
      maxRetries = 3,                             // 1 call + 3 retries
      delay = 200, multiplier = 2)                // wait 200, 400, 800 ms
  SpeakerProfile profile(String name) {
    return speakers.get(name);   // the flaky speaker API
  }
}
```

// delete: spring-retry

Also in the box

+ RetryTemplate and RetryPolicy for code you can't annotate
+ A RetryListener to log or count every attempt
+ @ConcurrencyLimit caps how many calls run at once
+ Jitter, so retries don't all land at once

// part 2

HTTP clients

Feign goes. Interfaces stay.

// why now

Spring Cloud OpenFeign is feature complete.

// http interfaces

Describe the API as an interface

```java
// TodoService.java
@HttpExchange(url = "/todos", accept = "application/json")
public interface TodoService {
  @GetExchange("/")
  List<Todo> getAllTodos();
  @GetExchange("/{id}")
  Todo getTodoById(@PathVariable Long id);
  @PostExchange("/")
  Todo createTodo(@RequestBody Todo todo);
}
```

// delete: the boilerplate

Boot 3: wire every client by hand

-3 beans

// HttpConfig.java

```
- @Bean RestClient restClient(RestClient.Builder builder) {
-   return builder.baseUrl("https://api.github.com").build();
- }
- @Bean HttpServiceProxyFactory factory(RestClient client) {
-   return HttpServiceProxyFactory
-       .builderFor(RestClientAdapter.create(client)).build();
- }
- @Bean GitHub gitHub(HttpServiceProxyFactory factory) {
-   return factory.createClient(GitHub.class);
- }
```

// framework 7

@ImportHttpServices

One annotation. No factory beans.

```java
// HttpConfig.java
@Configuration
@ImportHttpServices(
    group = "github",
    types = GitHub.class)
class HttpConfig {}
```

// boot 4

Boot does the rest

// application.yaml

```yaml
spring:
  http:
    serviceclient:
      github:
        base-url: https://api.github.com
        read-timeout: 5s
```

// framework 7

Need more? Configure the group

```
@Configuration
@ImportHttpServices(group = "todos", types = {TodoService.class, PostService.class})
@ImportHttpServices(group = "github", types = {RepoService.class, IssueService.class})
class MultiApiConfig {
  @Bean
  RestClientHttpServiceGroupConfigurer groupConfigurer() {
    return groups -> groups.filterByName("github")
        .forEachClient((group, builder) -> builder
            .baseUrl("https://api.github.com")
            .defaultHeader("Accept", "application/vnd.github.v3+json"));
  }
}
```

// delete: openfeign

Feign goes

-1 dependency

// GitHub.java

```
- @FeignClient(name = "github",
-       url = "${github.url}")
  interface GitHub {
-   @GetMapping("/users/{id}")
+   @GetExchange("/users/{id}")
    User get(@PathVariable Long id);
  }
```

// delete: openfeign

Feign goes

-1 dependency

// GitHub.java

```
  interface GitHub {
+   @GetExchange("/users/{id}")
    User get(@PathVariable Long id);
  }
```

// part 2

API versioning

Versions are part of the mapping now.

// the shift

We could always version APIs.
Now it's built in, MVC and WebFlux.
No more custom filters.

// delete: your version filter

That 2021 filter goes

-1 file

// VersionFilter.java (deleted)

```
- class VersionFilter extends OncePerRequestFilter {
-    // read the X-API-Version header
-    // rewrite the path to /v2/...
-    // 140 lines nobody touches
- }
```

// UserController.java

```
+ @GetMapping(path = "/users", version = "2.0")
  List<UserV2> users() { ... }
```

// delete: your version filter

That 2021 filter goes

-1 file

```
// VersionFilter.java  (deleted)

// UserController.java
+ @GetMapping(path = "/users", version = "2.0")
  List<UserV2> users() { ... }
```

// framework 7

Same path, two versions

```
// TalkController.java
@RequestMapping("/api/talks")
class TalkController {
  @GetMapping(version = "1.0")
  List<Talk> findAll() {
    return talks.findAll();
  }
  @GetMapping(version = "2.0")
  List<TalkSummary> findAllSummaries() {
    return talks.findAll().stream().map(TalkSummary::of).toList();
  }
}
```

// boot 4

One property picks the strategy

// application.yaml

```yaml
spring:
  mvc:
    apiversion:
      use:
        header: X-API-Version
      supported: 1.0,2.0
      default: "1.0"
```

// baseline versions

The "2.0+" trick

Don't copy endpoints that didn't change.

```
// TalkController.java
@GetMapping(version = "1.0")
List<Talk> findAll() { ... }

@GetMapping(version = "2.0+")   // 2.0, 2.1, 2.2 ...
List<TalkSummary> findAllSummaries() { ... }
```

// deprecation

Retire a version, politely

Boot picks up the bean. No WebMvcConfigurer.

```java
// ApiVersionConfig.java
@Bean
ApiVersionDeprecationHandler deprecations() {
  var handler = new StandardApiVersionDeprecationHandler();
  handler.configureVersion("1.0")
      .setDeprecationDate(ZonedDateTime.parse("2026-11-12T00:00Z"))
      .setSunsetDate(ZonedDateTime.parse("2027-06-30T00:00Z"))
      .setSunsetLink(URI.create("https://example.com/talks/v2"));
  return handler;
}
```

// rfc 9745, on every 1.0 response

The client sees the warning

```
// GET /api/talks, X-API-Version: 1.0
HTTP/1.1 200
Deprecation: @1794441600
Sunset: Wed, 30 Jun 2027 00:00:00 GMT
Link: <https://example.com/talks/v2>; rel="sunset"; type="text/html"
```

```
// delete: your version filter
```

It does more than routing

+ Header, query parameter, path or media type
+ A 400 for a version you don't support
+ Your own formats, like v1, with an ApiVersionParser
+ Clients send versions too, with RestClient

LIVE

```
$ git checkout demo-02-start
```

Demolition, live

Delete what Boot now does for you. Tests stay green.

// part 2

Testing

Fewer test clients, no WebFlux.



// framework 7

Meet RestTestClient

WebTestClient's fluent API, for Spring MVC. No WebFlux.

```
// ApiTests.java
@Autowired RestTestClient client;   // with @AutoConfigureRestTestClient

@Test
void speakerProfileSurvivesAFlakyApi() {
  client.get().uri("/api/talks/1/speaker")
      .exchange()
      .expectStatus().isOk()
      .expectBody()
      .jsonPath("$.name").isEqualTo("Dan Vega");
}
```

// delete: extra test clients

Test scaffolding goes

-1 dependency

// TodoControllerTest.java

```
- mockMvc.perform(get("/todos"))
-    .andExpect(status().isOk());
+ client.get().uri("/todos")
+    .exchange()
+    .expectStatus().isOk()
+    .expectBody(Todo[].class);
```

// pom.xml

```
- spring-boot-starter-webflux    <!-- only for WebTestClient -->
```

// delete: extra test clients

Test scaffolding goes

-1 dependency

// TodoControllerTest.java

```
+ client.get().uri("/todos")
+    .exchange()
+    .expectStatus().isOk()
+    .expectBody(Todo[].class);
```

```
// delete: extra test clients
```

One client, every kind of test

+ bindToController: a unit test, no Spring
+ bindTo(mockMvc): your slice tests
+ bindToApplicationContext: the full context
+ bindToServer: a real HTTP server

// part 2

Observability

OpenTelemetry, built in.

// delete: your observability setup

OpenTelemetry, one starter

```
- -javaagent:opentelemetry-javaagent.jar     // version headaches
- opentelemetry-spring-boot-starter          // third party, alpha
- spring-boot-starter-actuator + bridges     // more than you need
+ spring-boot-starter-opentelemetry          // OTLP built in
```

// delete: your observability setup

OpenTelemetry, one starter

```
+ spring-boot-starter-opentelemetry          // OTLP built in
```

// spring-boot-starter-opentelemetry

What you get out of the box

+ Traces for every request and RestClient call
+ Metrics, exported over OTLP
+ Trace and span IDs in every log line
+ Sampling defaults to 10%. Turn it up for local dev.
+ LGTM in Docker Compose. Boot wires it up.

// grafana/otel-lgtm

LGTM, letter by letter

L **Loki.** Your logs, searchable.

G **Grafana.** The screen you look at.

T **Tempo.** Your traces, request by request.

M **Mimir.** Your metrics, Prometheus style.

One container. Open localhost:3000 and it's all there.

// the diffstat, one demo app

What left your pom

// pom.xml

```
- <artifactId>spring-retry</artifactId>
- <artifactId>spring-cloud-starter-openfeign</artifactId>
- <artifactId>spring-boot-starter-webflux</artifactId>
```

// VersionFilter.java (deleted)

Three dependencies and one filter.

All built into Framework 7.

−3

Part 3

Small things, quietly useful

null safety, BeanRegistrar, and six more

// part 3

Null safety

JSpecify, across the portfolio.

Ever hit a NullPointerException?

Yeah. Everyone.

Every single one of you.

// the real problem

The problem isn't null.
It's that nullness is implicit.
Undocumented.
The compiler can't help.

// jspecify.dev

Four annotations, one standard

`@Nullable`

This value can be null. A field, return, parameter or generic.

`@NonNull`

Null is never valid here.

`@NullMarked`

Everything in this scope is non-null by default.

`@NullUnmarked`

Nullness is unspecified. Opt a scope back out.

// delete: org.springframework.lang

Spring's annotations go away

-4 annotations

```
// package-info.java
- @NonNullApi
+ @NullMarked
  package dev.danvega.talks;
- import org.springframework.lang.NonNullApi;
+ import org.jspecify.annotations.NullMarked;
// Talk.java
- import org.springframework.lang.Nullable;
+ import org.jspecify.annotations.Nullable;
```

// delete: org.springframework.lang

Spring's annotations go away

-4 annotations

```
// package-info.java
+ @NullMarked
  package dev.danvega.talks;
+ import org.jspecify.annotations.NullMarked;
// Talk.java
+ import org.jspecify.annotations.Nullable;
```

// boot 4

JSpecify null safety

Null becomes part of the type.

```
// package-info.java
@NullMarked
package dev.danvega.orders;

// OrderService.java
@Nullable Order find(String id);
```

// the payoff

Spring's own APIs now tell you where null can come back.

// part 3

Programmatic beans

BeanRegistrar replaces the post-processor.

// delete: the post-processor

Boot 3: register beans by hand

// Feeds.java

```
- class Feeds implements BeanDefinitionRegistryPostProcessor {
-   public void postProcessBeanDefinitionRegistry(BeanDefinitionRegistry registry) {
-     for (String name : List.of("devoxx", "jfall")) {
-       var definition = BeanDefinitionBuilder
-           .genericBeanDefinition(Feed.class, () -> new Feed(name))
-           .getBeanDefinition();
-       registry.registerBeanDefinition(name + "Feed", definition);
-     }
-   }
-   public void postProcessBeanFactory(ConfigurableListableBeanFactory factory) {}
- }
```

// framework 7

BeanRegistrar

```
// @Import(Feeds.class) on any @Configuration
class Feeds implements BeanRegistrar {
  public void register(BeanRegistry registry, Environment env) {
    for (String name : List.of("devoxx", "jfall")) {
      registry.registerBean(name + "Feed", Feed.class,
          spec -> spec.supplier(ctx -> new Feed(name)));
    }
    if (env.matchesProfiles("dev")) {
      registry.registerBean("mirror", Mirror.class, spec -> spec.lazyInit()
          .supplier(ctx -> new Mirror(ctx.bean("jfallFeed", Feed.class))));
    }
  }
}
```

// easy to miss

Six small things

Spring AI 2

Built on Boot 4

JmsClient

Like JdbcClient, for JMS

Security MFA

Multi-factor login, built in

Data AOT

Faster startup

SSRF guard 4.1

Block calls you don't allow

Lazy JDBC 4.1

Connect on first statement

Part 4

The upgrade plan

upgrade first, delete second

// your plan

Five steps

1. Get to the latest 3.5.x first
2. Bump to Boot 4.1 and run the tests
3. Fix Jackson, then starters, then tests
4. Check what went quiet: Flyway, consoles, metrics
5. Then delete what Boot now does for you

// your plan

Let tools do the boring part

> OpenRewrite has Boot 4 recipes

> Spring **Application Advisor** plans the order

> Your tests are the real safety net

// recap



Delete the extras



Find the quiet breaks



Stay supported

// wrap-up

Resources

Migration guide

github.com/spring-projects/spring-boot/wiki

Blog posts

danvega.dev/blog

Demo code

github.com/danvega/spring-cleaning

Spring Boot 4 playlist

youtube.com/@danvega · 18 videos



Scan for the code

```
$ git commit -m "spring cleaning"
```

Thank you

Questions?

danvega.dev @therealdanvega github.com/danvega